Informe Unity -



CatDeffense

Primer Trabajo: Animación Digital.

Realizado por: Alberto García Martín (70965216G)



VNiVERSiDAD D SALAMANCA

VISIÓN GENERAL **2**
SPRITES Y PALETAS **2**
VIDEOS DIDÁCTICOS **2**
MÚSICA **2**
EFECTOS DE SONIDO **2**
OTROS **2**
INFORME **3**
1. Fundamentos y prototipado. 3
1.1. Creación del mundo. 3
1.2. La entidad base. 7
1.3. Implementación del jugador. 8
2. Implementación del “Core Loop”. 13
2.1. El objetivo. 13
2.2. Los enemigos. 14
2.3. La interacción. 16
3. Creación del sistema de juego. 16
3.1. Sistema de puntuación. 16
3.2. Sistema de recolección. 17
3.3. Sistema de generación. 19
4. Flujo de Juego. 22
4.1. El cerebro. 22
4.2. El flujo de pantalla. 23
4.2. El ciclo completo. 23
5. Pulido y resolución de problemas. 28
5.1. Bugs más notorios. 28
5.2. Pulido y elementos de sonido. 28
CONCLUSIONES **29**

VISIÓN GENERAL

Unity. Será necesario hacer un videojuego a partir de los ejercicios hechos en clase o bien desde cero según lo prefiera el usuario.

SPRITES Y PALETAS

PERSONAJE PRINCIPAL
MUNDO1
GOBBLIN
COINS
OTP
MUNDO2
NECROMANCER

VIDEOS DIDÁCTICOS

- Cómo crear un fondo con animación en Unity (Efecto Parallax en Unity)
- Learn Unity by Making a 2D Game – Full Beginner Course (6 Hours)
- PLEASE use a Unity SOUND MANAGER! - Full Tutorial
- How to do Unity Scene Transition

MÚSICA

Toda la música se inspiró en el juego Just Shapes & Beats.
https://www.youtube.com/playlist?list=PLydwAuTBfPJX0dn9OrrHpH0F3hjmPRuXe

EFECTOS DE SONIDO

https://pixabay.com/es/sound-effects/retro-coin-4-236671/
https://pixabay.com/es/sound-effects/goblin-death-6729/
https://pixabay.com/es/sound-effects/little-creature-hurt-sound-295405/
Todos los restantes se pueden encontrar aquí:
https://pixabay.com/sound-effects/

OTROS

Para las imágenes del menú, resolución de problemas y apoyo en algunos objetivos, se ha hecho uso de Gemini AI.

INFORME

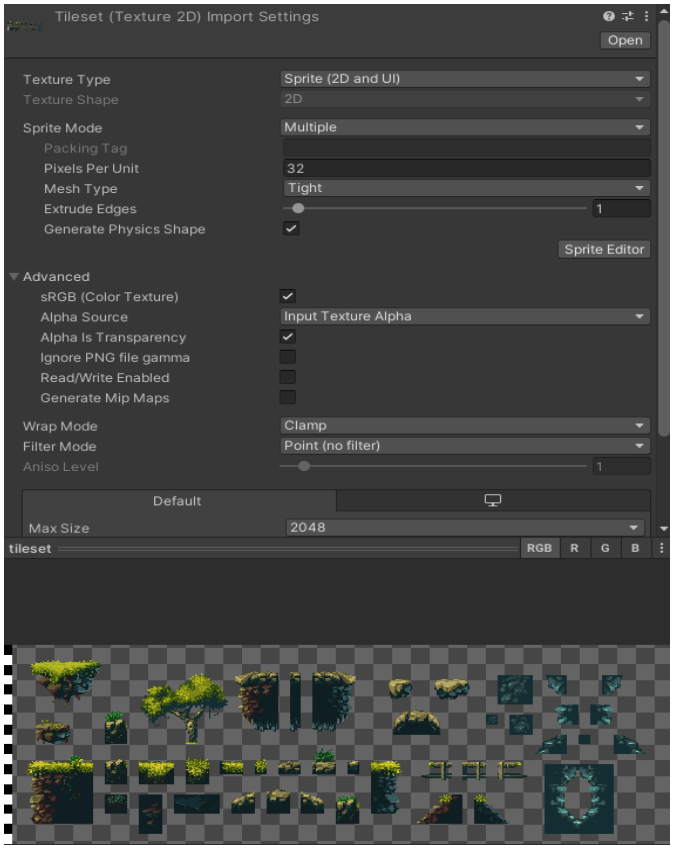
1. Fundamentos y prototipado.

Lo primero que se ha realizado es el establecimiento de un entorno de juego para posteriormente agregar una entidad jugable con la que el usuario pudiese interactuar. El diseño del código está pensado para su escalabilidad, comenzando con una clase de entidad genérica.

1.1. Creación del mundo.

Para empezar creamos una nueva escena (World1). Creamos un Empty Object Level1 que contendrá el fondo y los elementos correspondientes a la escena.



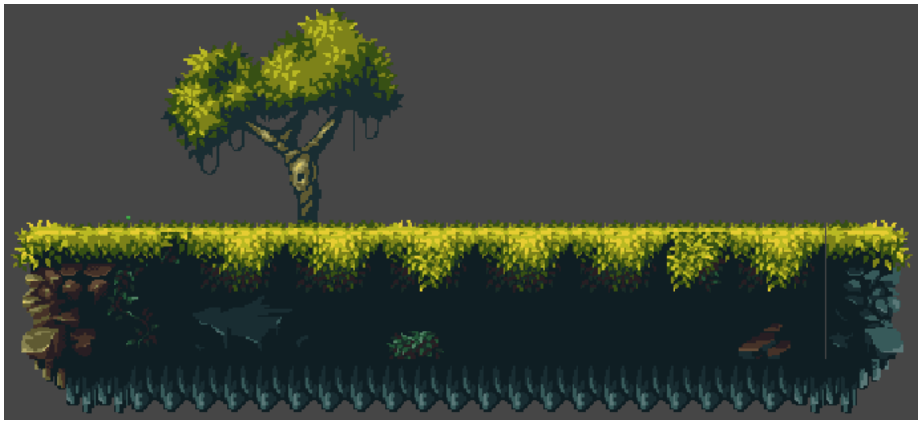


En esta escena crearemos varios Tilemaps: BackEnvironment, FrontEnvironment y Floor, ya que con ellos crearemos el escenario del jugador.

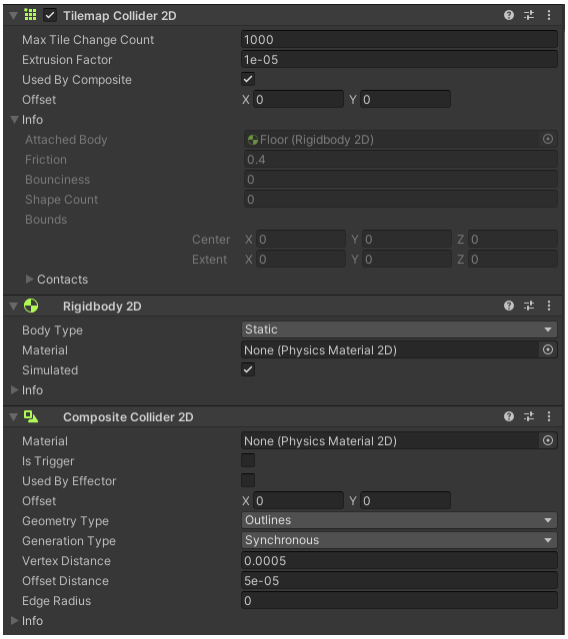
Lo siguiente es coger nuestro Tile Palette descargado, ajustamos sus parámetros y editamos en sprite editor, realizamos un slice de 56x20 ya que son las medidas para cada píxel del mapa. arrastramos hacia nuestro Tile Palette el elemento ya cortado y lo agregamos en una carpeta llamada W1, que contendrá todos los elementos para pintar el escenario. Lo siguiente es pintar este mismo escenario, para ello hay que tener en cuenta de que manera se quiere tener ordenados en pantalla y superpuestos los distintos elementos, es por ello que se crea un sorting layer:

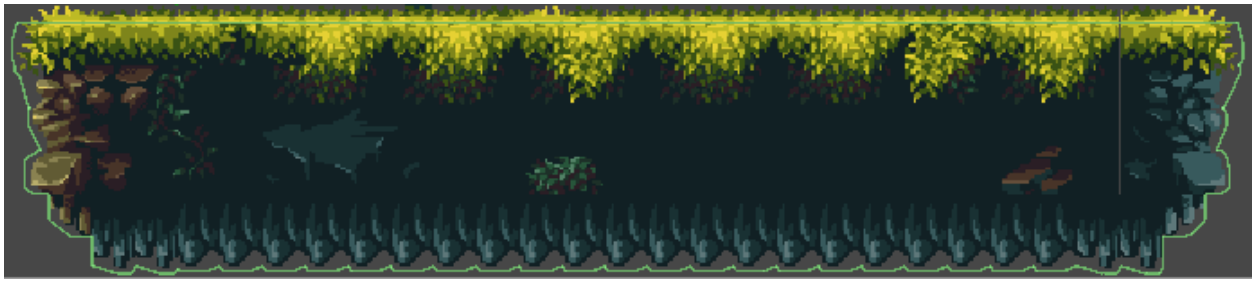


Que va a referenciar donde queremos posicionar cada paleta, si queremos que esté en frente del jugador la pintaremos en Front, en cambio, si queremos elementos que están detrás, en Back, y los elementos que sean colliders, en Floor. El resultado es el siguiente:



Para crear los colliders del suelo, se ha creado un tilemap collider junto con un RigidBody en estático y un Composite Collider, esto era debido a que el personaje no referencia correctamente el suelo al realizar el Build del proyecto:
TileMap lo marco como que está siendo usado por Composite, mientras que el rigidbody lo pongo en tipo estático.





Lo siguiente será crear el fondo, para ello utilizaremos una serie de elementos y la combinaremos para formar un parallax estático. Para ello:

1. Se crea un Empty Object con Background.
2. Dentro de ese elemento se añade todo el fondo dividido en sets.
3. Cada uno de estos elementos (Sea, Cloud o Sky) tiene un sorting layer correspondiente.
4. Cada elemento tiene un propio Script dedicado al movimiento del fondo.

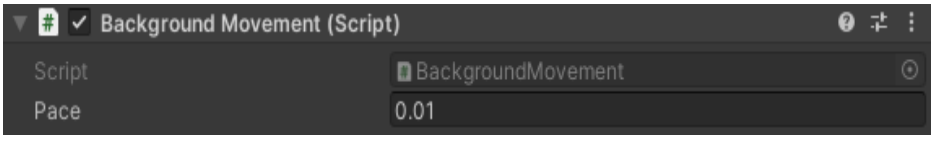



```
using UnityEngine;

public class BackgroundMovement : MonoBehaviour
{
    private Material material2;
    public float pace = 0.1f;

    private void Awake()
    {
        material2 = GetComponent<SpriteRenderer>().material;
    }

    void Update()
    {
        material2.SetTextureOffset("_MainTex", new Vector2(Time.time * pace, 0));
    }
}
```



El resultado es el siguiente:



1.2. La entidad base.

Como tal vamos a trabajar con entidades, estas entidades son elementos de nuestro juego que pueden interaccionar de alguna u otra manera con el jugador, elementos “vivos” de nuestra escena, ya sean enemigos, personajes o elementos a proteger.

En nuestro caso, vamos a crear enemigos, un elemento a proteger (en este caso, un gato), y un personaje principal, que será el que controlaremos.

Cuando se crea un script de Entidad se debe de tener en cuenta que acciones puede hacer cualquier elemento vivo, ya sea girar, moverse, correr, andar, atacar, morir, etc.

En mi caso comencé creando el script de Player, pero a medida que se avanzó y con motivos de refinamiento para añadir más entidades, se acabó creando este script general, y los demás scripts, ya sea Player, Enemy o OTP, heredan de esta clase general.

Las responsabilidades clave de Entity.cs son:

- Componentes centrales: almacenamiento de referencias a RigidBody2D, Animator, Collider2D y SpriteRenderer.
- Salud: gestión de la lógica de vida, incluyendo entre estas maxHealth, currentHealth y un método virtual Die().
- Lógica de Físicas: configuración de isGrounded utilizando un Raycast y la capacidad de girar con Flip().
- Interacción: definir métodos de combate, DamageTargets() y TakeDamage(int amount).

1.3. Implementación del jugador.

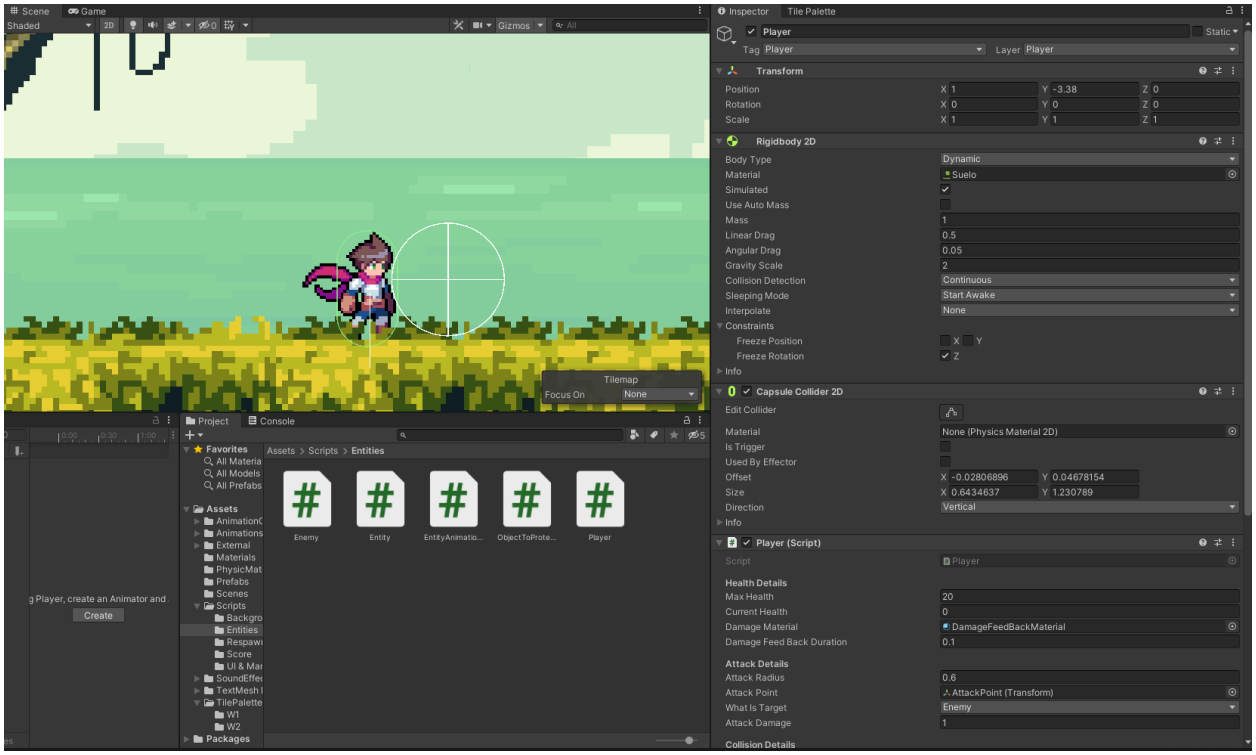
Una vez definida la base, Player solo hereda de Entity, pudiéndose centrar únicamente en lo que hace único al jugador:

1. Control por teclado: se implementó el método HandleInput() para leer las entradas del usuario.
2. Movimiento específico: se sobreescribe HandleMovement() para darle al jugador un control directo sobre su velocidad, implementando un walkSpeed y un runSpeed dependiendo de la tecla “Shift”.
3. Salto: habilidad única en el jugador. Se añade el método TryToJump(), que aplica una jumpForce al Rigidbody2D solo si isGrounded es verdadera.

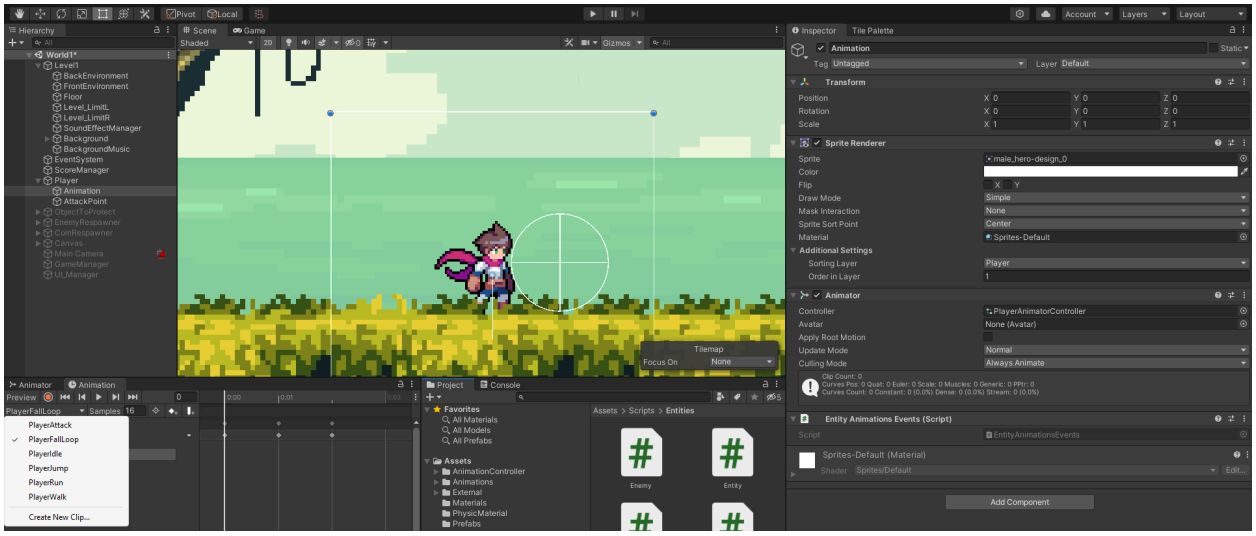
Se crea un Empty Object llamado Player, con un tag nuevo “Player” y layer Player. Dentro de este objeto estarán otros dos objetos, uno para las animaciones y otro para el punto de ataque.



Dentro de Player incluimos el script de Player que hereda de Entity, además de asignarle un RigidBody2D y un CapsuleCollider2D ajustado a su tamaño.



Posteriormente en Animator, se añade un SpriteRenderer2D junto con un Animator y su correspondiente PlayerController. Además se le añade otro Script llamado EntityAnimationsEvents que nos permite asignar eventos a la animación.



Estos eventos se corresponden con el método de ataque, una vez golpee el enemigo se llamará a DamageTargets(). EnableMovement lo que hará será detener el movimiento del jugador si está

atacando, esto se hace para que no se pueda mover y atacar a la vez y que haya un cooldown de un ataque a otro.

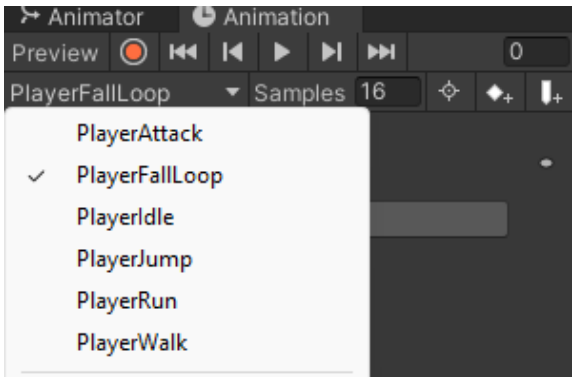
```
using UnityEngine;

public class EntityAnimationsEvents : MonoBehaviour
{
    private Entity entity;
    private void Awake()
    {
        entity = GetComponentInParent<Entity>();
    }

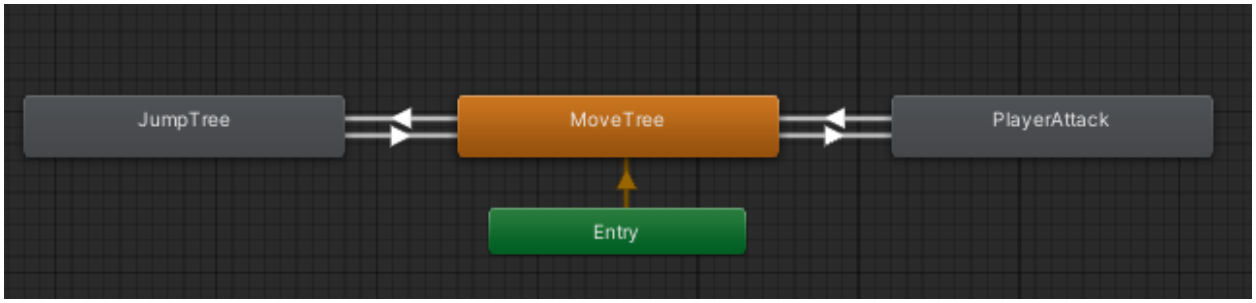
    public void DamageTargets() => entity.DamageTargets();
    public void DisableMovementAndJump() => entity.EnableMovement(false);
    public void EnableMovementAndJump() => entity.EnableMovement(true);

}
```

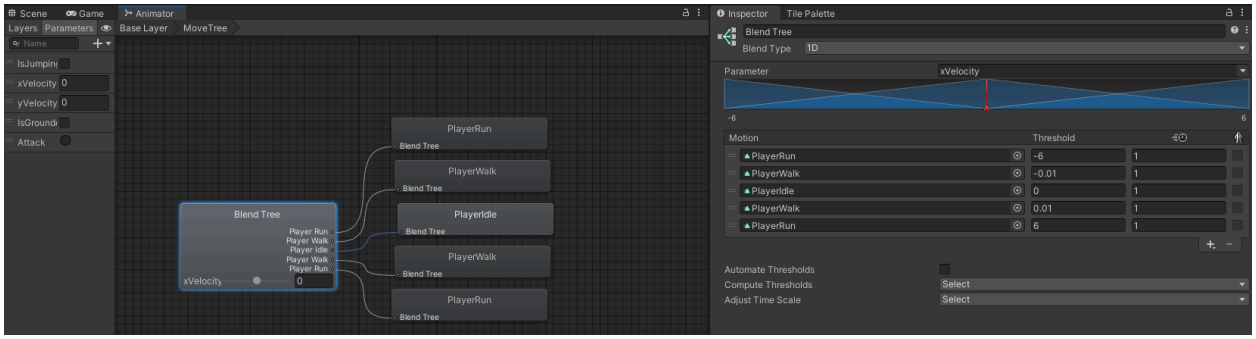
En animator, se añaden todas las animaciones que se van a utilizar.



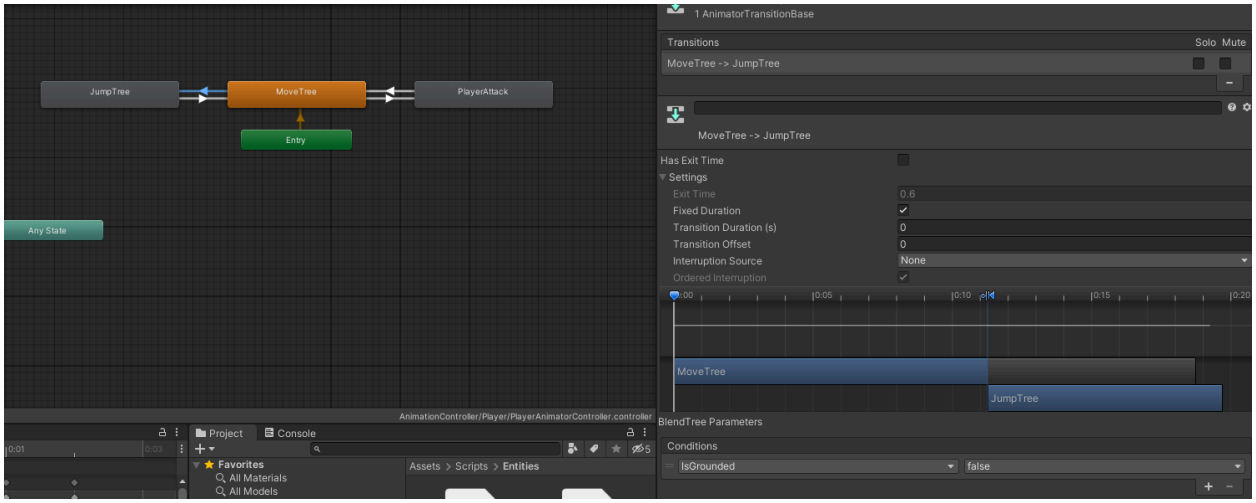
Como tal solo explicaré este Animator Controller, ya que los demás son iguales o incluso menos. El jugador tiene 2 Blend Trees que corresponden al movimiento y al salto, junto con otra animación separada al salto que es la de ataque (no se va a atacar mientras isGrounded sea false).

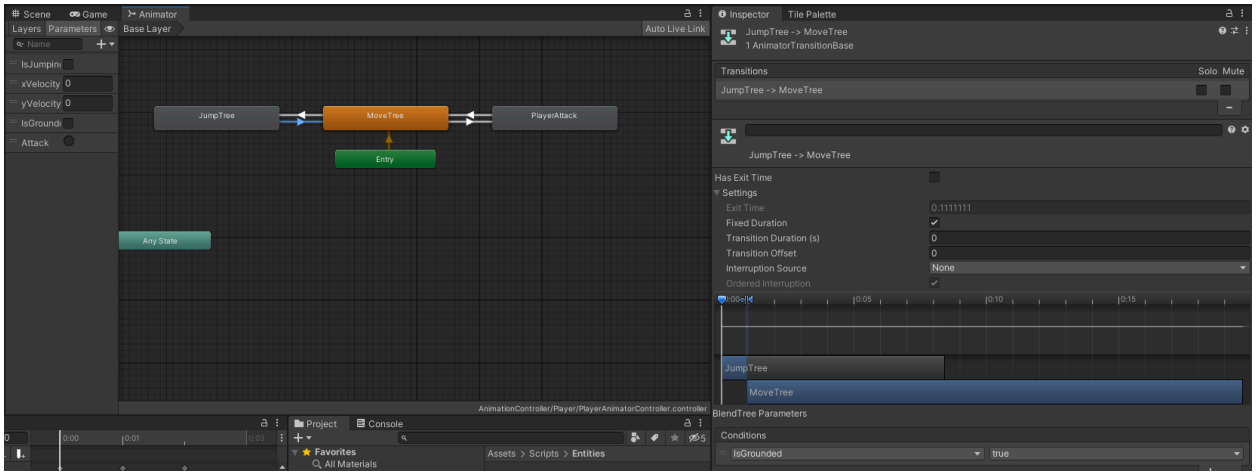


Dentro de moveTree tendremos nuestras velocidades asignadas para cambiar de una animación a otra dependiendo del xVelocity:

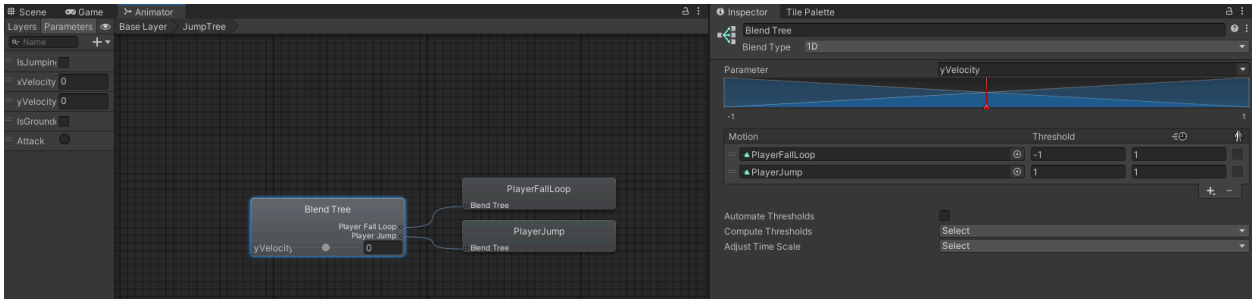


En cuanto al salto, la transición depende de isJumping:

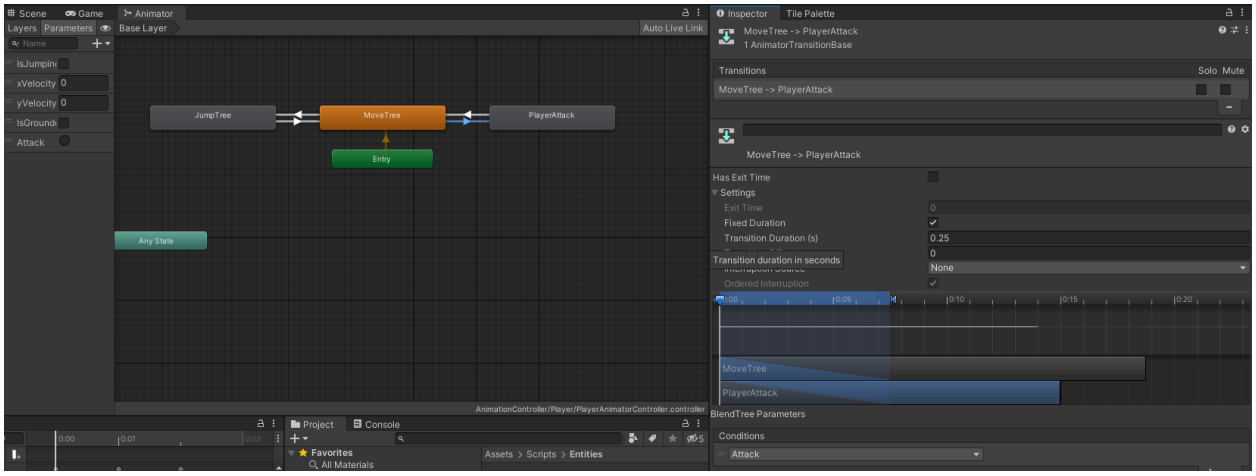




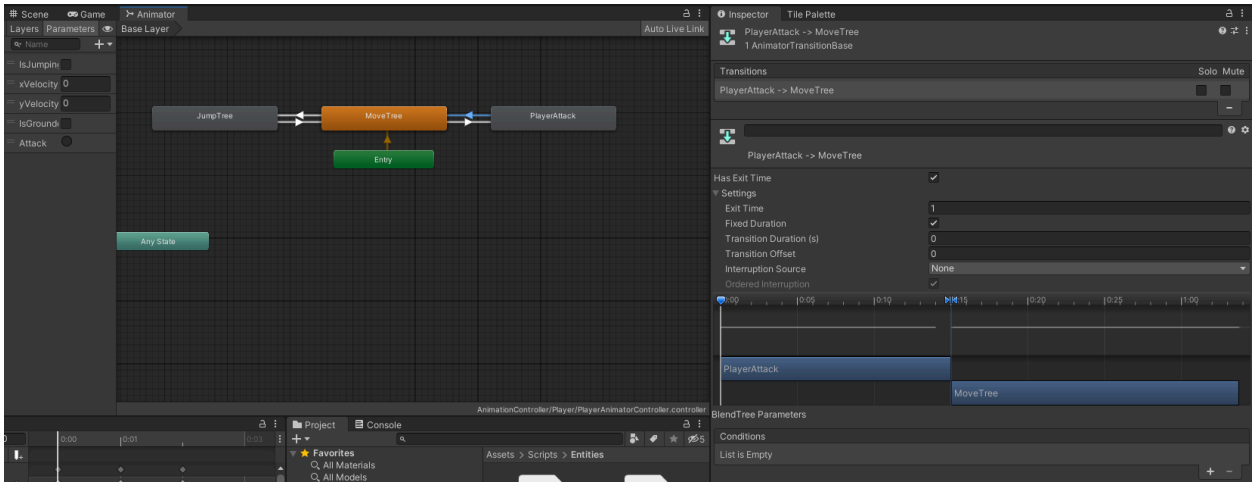
Dentro del Tree de salto, hay 2 animaciones dependiendo de yVelocity, salto y caída:



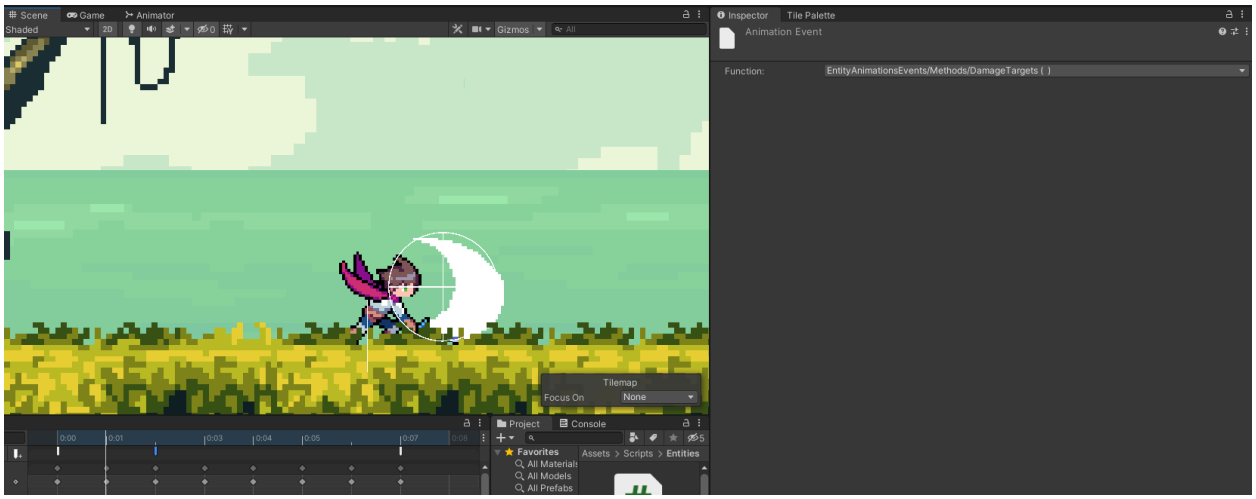
Finalmente para atacar:



La transición de vuelta tiene Exit Time ya que no tiene ninguna condición de vuelta a normalidad, debe de salir inmediatamente después de realizar la animación:



En cuanto a la animación de ataque se añaden los 3 eventos que permiten parar al jugador y dañar a los enemigos:



El daño a los enemigos depende el EmptyObject AttackPoint que contiene un Gizmo que sirve como área de daño aplicado, justo al alcanzar el frame de ataque, se añade el evento.
Para los enemigos es parecido, solo que en este caso no hay salto, ni movimiento de andar.
Para el OTP, se ha hecho únicamente la animación Idle, ya que estará estático en el juego.

2. Implementación del “Core Loop”.

2.1. El objetivo.

Con las bases establecidas, el siguiente paso fue implementar el ciclo de juego principal. Este ciclo es definido por la tensión entre proteger al gato y los antagonistas (los enemigos que van a intentar destruirlo), con el jugador como principal defensor.

```
using UnityEngine;

public class ObjectToProtect : Entity
{
    [Header("Extra Details")]
    private Transform player;
    [SerializeField] private AudioClip[] hurtSoundClips;

    protected override void Awake()
    {
        base.Awake();
        player = FindFirstObjectByType<Player>().transform;
    }
    protected override void Update()
    {
        HandleFlip();
    }
    protected override void HandleFlip()
    {
        if (player == null)
            return;

        if (player.transform.position.x > transform.position.x && facingRight == false)
            Flip();
        else if (player.transform.position.x < transform.position.x && facingRight ==
true)
            Flip();
    }

    protected override void Die()
    {
        base.Die();
        GameManager.instance.EnableGameOverUI();
    }

    protected override void TakeDamage(int amount)
    {
        base.TakeDamage(amount);
        SoundFXManager.Instance.PlayRandomSFXClip(hurtSoundClips, transform,
0.25f);
    }
```

El corazón del juego y la condición de derrota principal es la entidad a proteger u OTP:

1. Hereda de Entity: se le asigna automáticamente la función de salud y la capacidad de recibir daño.
2. Lógica de derrota: la funcionalidad principal es la anulación del método Die. Cuando la salud del gato llega a 0, se ejecuta el método Die() personalizado, que, además de la lógica base de Entity, ejecutará el final del juego.
3. Interacción con Jugador: para hacerlo sentir más vivo, se sobreescribe HandleFlip() para que a diferencia de la entidad base que gira por velocidad, este gire buscando la posición del jugador.

2.2. Los enemigos.

Los enemigos son la fuerza principal contra la que el jugador reacciona. Se han creado dos entidades de enemigos, Goblin y Necromancer, uno para cada nivel.

1. Herencia y movimiento: hereda de Entity, su movimiento básico sobreescribe el HandleMovement(), donde se aplica una velocidad constante en la dirección en la que esté mirando.

2. IA de detección: la inteligencia del enemigo reside en HandleCollision(). Además de llamar a la comprobación de suelo, el enemigo utiliza, al igual que el jugador, un attackPoint para escanear objetivos.
3. Lógica de ataque y giro:
    a. El método HandleAttack() es activado si playerDetected es verdadero, disparando el trigger “Attack” del animador.
    b. Para solucionar un error que provocaba que el enemigo ataque de espaldas, se sobreescribe HandleFlip() para hacer lo mismo que el gato. De primeras, el enemigo buscará al jugador, si encuentra al OTP antes, lo atacará, ya que corresponde al mismo target.

```
[Header("Extra Details")]
[SerializeField] private AudioClip[] damageSoundClips;

protected override void Awake()
{
    base.Awake();
    player = FindFirstObjectByType<Player>()?.transform;
}

protected override void Update()
{
    base.Update();
    HandleAttack();
}
protected override void HandleMovement()
{
    if (canMove)
        rb.velocity = new Vector2(facingDir * walkSpeed, rb.velocity.y);
    else
        rb.velocity = new Vector2(0, rb.velocity.y);
}

protected override void HandleFlip()
{
    if (player == null)
    {
        base.HandleFlip();
        return;
    }

    if (player.transform.position.x > transform.position.x && facingRight == false)
        Flip();
    else if (player.transform.position.x < transform.position.x && facingRight ==
true)
        Flip();
}


protected override void HandleAttack()
{
    if (playerDetected)
    {
        anim.SetTrigger("Attack");
    }

}

protected override void HandleCollision()
{
    base.HandleCollision();
    playerDetected = Physics2D.OverlapCircle(attackPoint.position,
attackRadius, whatIsTarget);
}

protected override void TakeDamage(int amount)
{
    base.TakeDamage(amount);
    SoundFXManager.Instance.PlayRandomSFXClip(damageSoundClips,
transform, 0.25f);
}
}
```

2.3. La interacción.

El core loop se cierra cuando estas entidades interactúan entre sí mediante el combate. El sistema fue diseñado para ser flexible y desacoplarse de las animaciones.

1. Inicio de ataque: cuando el jugador pulsa para atacar o la IA del enemigo decide atacar, se activa el trigger de ataque.
2. Daño por evento de animación: el daño no se aplica instantáneamente, en su lugar se utiliza un evento en el fotograma clave, DamageTargets() es la resultante de la creación de un círculo en la posición del attackPoint de la entidad, filtra los objetos golpeados usando la layermask whatIsTarget, para cada objetivo golpeado, obtiene su componente y llama al método TakeDamage() pasándole el attackDamage del atacante.

```
protected override void TakeDamage(int amount)
{
    base.TakeDamage(amount);
    SoundFXManager.Instance.PlayRandomSFXClip(damageSoundClips,
transform, 0.25f);
}
```

3. Creación del sistema de juego.

3.1. Sistema de puntuación.

La base de todos los sistemas fue el ScoreManager. Se diseñó como un Singleton que gestiona la puntuación de toda la partida.

1. Diseño: utiliza un patrón de instance estática en su awake, se garantiza siempre que solo haya un gestor de puntuación.
2. Persistencia: se llama a DontDestroyOnLoad(gameObject), permite que el ScoreManager sobreviva intacto al cambiar de nivel.
3. Puntos: expone un método AddScore(int amount), cuando se recoge una moneda, se añade ese valor a la puntuación.
4. Eventos: OnScoreChanged permite desacoplar el ScoreManager de la UI. Cuando la puntuación cambia, dispara el evento. Otros scripts como ScoreUI y GameManager pueden subscribirse a este evento para reaccionar al cambio.

```
using TMPro;
using UnityEngine;

public class ScoreUI : MonoBehaviour
{
    [SerializeField] private TextMeshProUGUI scoreText;

    private void Start()
    {
        if (ScoreManager.Instance != null)
        {
            scoreText.text = ScoreManager.Instance.Score.ToString();
            ScoreManager.Instance.OnScoreChanged += UpdateScore;
        }
    }

    private void OnDestroy()
    {
        if (ScoreManager.Instance != null)
            ScoreManager.Instance.OnScoreChanged -= UpdateScore;
    }

    private void UpdateScore(int newScore)
    {
        scoreText.text = newScore.ToString();
    }
}
```

3.2. Sistema de recolección.

Las monedas son el principal medio para obtener puntuación.

1. Detección: Coin utiliza OnTriggerEnter2D para detectar las colisiones, comprobando si esa colisión tiene la etiqueta Player.
2. Valor: el valor depende del enum { Ruby, Gold, Silver } que indica el tipo de moneda y su correspondiente valor (10, 5 y 2).
3. Recolección: al ser recogida, se llama a AddScore(value), pasando el valor de su tipo.
4. Autodestrucción: para que el nivel no se llene de monedas, estas tienen un lifeTime en su método Init, llama a Invoke con la función SelfDestruct y un tiempo lifeTime. Si la moneda es recogida, este invoke es cancelado, si no, esta moneda se autodestruye pasado un tiempo específico.

```
    public void Init(CoinRespawner respawner, int spawnIndex)
    {
        this.respawner = respawner;
        this.spawnIndex = spawnIndex;
        Invoke(nameof(SelfDestruct), lifeTime);
    }

    private void OnTriggerEnter2D(Collider2D collision)
    {
        if (GameManager.instance != null && GameManager.instance.IsGameOver)
            return;

        if (collision.CompareTag("Player") && !isCollected)
        {
            isCollected = true;
            CollectCoin();
        }
    }

    private void CollectCoin()
    {
        if (SoundFXManager.Instance != null && coinSoundClip != null)
            SoundFXManager.Instance.PlaySFXClip(coinSoundClip, transform, 0.75f);

        int value = GetCoinValue();
        ScoreManager.Instance.AddScore(value);

        CancelInvoke(nameof(SelfDestruct));
        Destroy(gameObject);
    }

    private int GetCoinValue()
    {
        switch (coinType)
        {
            case CoinType.Ruby: return 10;
            case CoinType.Gold: return 5;
            case CoinType.Silver: return 2;
            default: return 1;
        }
    }

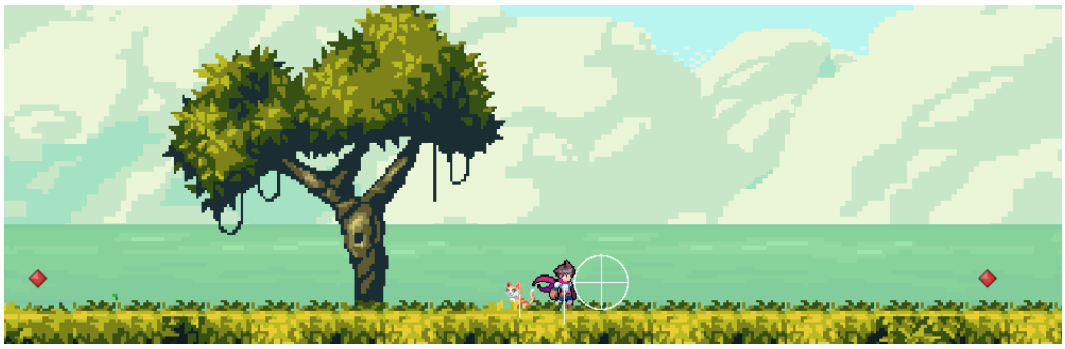
    private void SelfDestruct()
    {
        if (!isCollected)
        {
            Destroy(gameObject);
        }
    }

    private void OnDestroy()
    {
        if (respawner != null)
        {
            respawner.FreeSpawnPoint(spawnIndex);
        }
    }
}
```

3.3. Sistema de generación.

Estos scripts funcionan como directores de escena. Ambos generadores comprueban el estado del GameManager para detener la generación de enemigos/monedas en caso de que la partida haya terminado (es decir, IsGameOver).

1. EnemyRespawner:
    a. Generación por cooldown: utiliza un timer que se va descontando con Time.deltatime. Cuando llega a 0, genera un enemigo en un respawnPoint aleatorio y reinicia el timer.
    b. Dificultad progresiva: para aumentar el desafío, el cooldown se va reduciendo ligeramente cada vez que un enemigo es generado, haciendo que los enemigos aparezcan cada vez más rápido.



```
private void Start()
{
    ResetRespawner();
}

private void Awake()
{
    initialCooldown = cooldown;
}

private void Update()
{

    if (GameManager.instance != null && GameManager.instance.IsGameOver)
        return;

    timer -= Time.deltaTime;

    if (timer <= 0)
    {
        timer = cooldown;
        CreateNewEnemy();
        cooldown = Mathf.Max(coolDownCap, cooldown - coolDownDecreaseRate);
    }

}

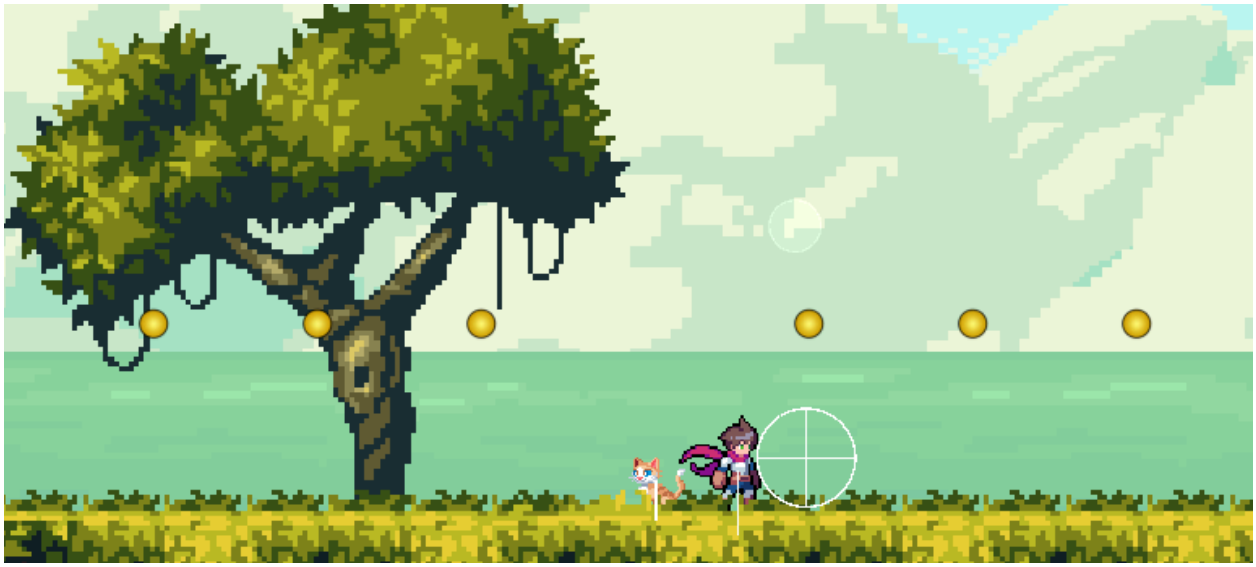
private void CreateNewEnemy()
{
    int respawnPointIndex = Random.Range(0, respawnPoints.Length);
    Vector3 spawnPoint = respawnPoints[respawnPointIndex].position;

    GameObject newEnemy = Instantiate(enemyPrefab, spawnPoint, Quaternion.identity);

    if (player != null)
    {
        bool createdOnTheRight = newEnemy.transform.position.x > player.transform.position.x;
        if (createdOnTheRight)
            newEnemy.GetComponent<Enemy>().Flip();
    }
}

public void ResetRespawner()
{
    timer = cooldown;
    player = FindFirstObjectByType<Player>()?.transform;
    cooldown = initialCooldown;
}
}
```

2. CoinRespawner:
    a. Gestión del espacio: este generador es más complejo, ya que no se pueden generar monedas en puntos ya ocupados, para solucionarlo, se crea una lista con las zonas ocupadas del campo de juego.
    b. Generación: cuando el timer llega a 0, busca en los spawnpoints libres, elige uno al azar y genera una moneda de un tipo aleatorio.
    c. Ciclo completo: al generar la moneda, le pasa su índice, cuando esa moneda es destruida, llama de vuelta al generador, liberando un espacio para las futuras monedas.



```
using System.Collections.Generic;
using UnityEngine;

public class CoinRespawner : MonoBehaviour
{
    [Header("Coin Prefabs")]
    [SerializeField] private GameObject goldCoinPrefab;
    [SerializeField] private GameObject silverCoinPrefab;
    [SerializeField] private GameObject rubyCoinPrefab;

    [Header("Spawn Settings")]
    [SerializeField] private Transform[] spawnPoints;

    [Header("Lifetime Settings")]
    [SerializeField] private float cooldown = 5f;


    private float timer;
    private List<int> occupiedPoints = new List<int>();

    private void Update()
    {

        if (GameManager.instance != null && GameManager.instance.IsGameOver)
            return;

        timer -= Time.deltaTime;

        if (timer <= 0)
        {
            timer = cooldown;
            SpawnRandomCoin();
        }
    }

    private void Start()
    {
        ResetRespawner();
    }
```

```
private void SpawnRandomCoin()
{
    List<int> freePoints = new List<int>();
    for (int i = 0; i < spawnPoints.Length; i++)
    {
        if (!occupiedPoints.Contains(i))
            freePoints.Add(i);
    }

    if (freePoints.Count == 0)
        return;

    int pointIndex = freePoints[Random.Range(0, freePoints.Count)];
    Vector3 spawnPos = spawnPoints[pointIndex].position;
    spawnPos.z = 0;

    GameObject prefabToSpawn = GetRandomCoinPrefab();
    GameObject coin = Instantiate(prefabToSpawn, spawnPos, Quaternion.identity);

    occupiedPoints.Add(pointIndex);

    Coin coinScript = coin.GetComponent<Coin>();
    if (coinScript != null)
        coinScript.Init(this, pointIndex);
}



private GameObject GetRandomCoinPrefab()
{
    int random = Random.Range(0, 3);
    switch (random)
    {
        case 0: return goldCoinPrefab;
        case 1: return silverCoinPrefab;
        case 2: return rubyCoinPrefab;
        default: return silverCoinPrefab;
    }
}

public void FreeSpawnPoint(int index)
{
    occupiedPoints.Remove(index);
}

public void ResetRespawner()
{
    timer = cooldown;
    occupiedPoints.Clear();
}
```

4. Flujo de Juego.

Una vez implementados los sistemas de combate y de puntuación, el objetivo era estructurar la experiencia en un ciclo completo. Esto requiere de un manejador que gestionará los estados globales, como la victoria y la derrota, y la transición entre escenas.

4.1. El cerebro.

El GameManager es el script más importante para el flujo de juego, diseñado como un Singleton persistente.

1. Patrón: al igual que ScoreManager, utiliza un patrón de instancia estática y llama a DontDestroyOnLoad(gameObject) en su awake con el fin de gestionar la partida moviéndose entre niveles.
2. Estado: mantiene estados clave como isGameOver e isLevelComplete:
    a. isGameOver: ¿el jugador ha perdido?
    b. isLevelComplete: ¿el jugador ha completado el nivel?

3. Condición de derrota: proporciona un método público EnableGameOverUI(). Este método se llama externamente por Player o OTP cuando uno de ellos muere. Su función es:
    a. Activa la bandera isGameOver.
    b. Reduce el tiempo para un efecto a cámara lenta.
    c. Activa el panel gameOverUI.
    d. Anula una posible victoria si el jugador muere después de ganar pero antes de pasar el nivel.
4. Condición de victoria: el GameManager se suscribe al evento OnScoreChanged, cuando este evento se dispara desde ScoreManager, comprueba si la puntuación ha alcanzado la meta, si es así, inicia la lógica de victoria (mostrar el panel “Press Q”).

4.2. El flujo de pantalla.

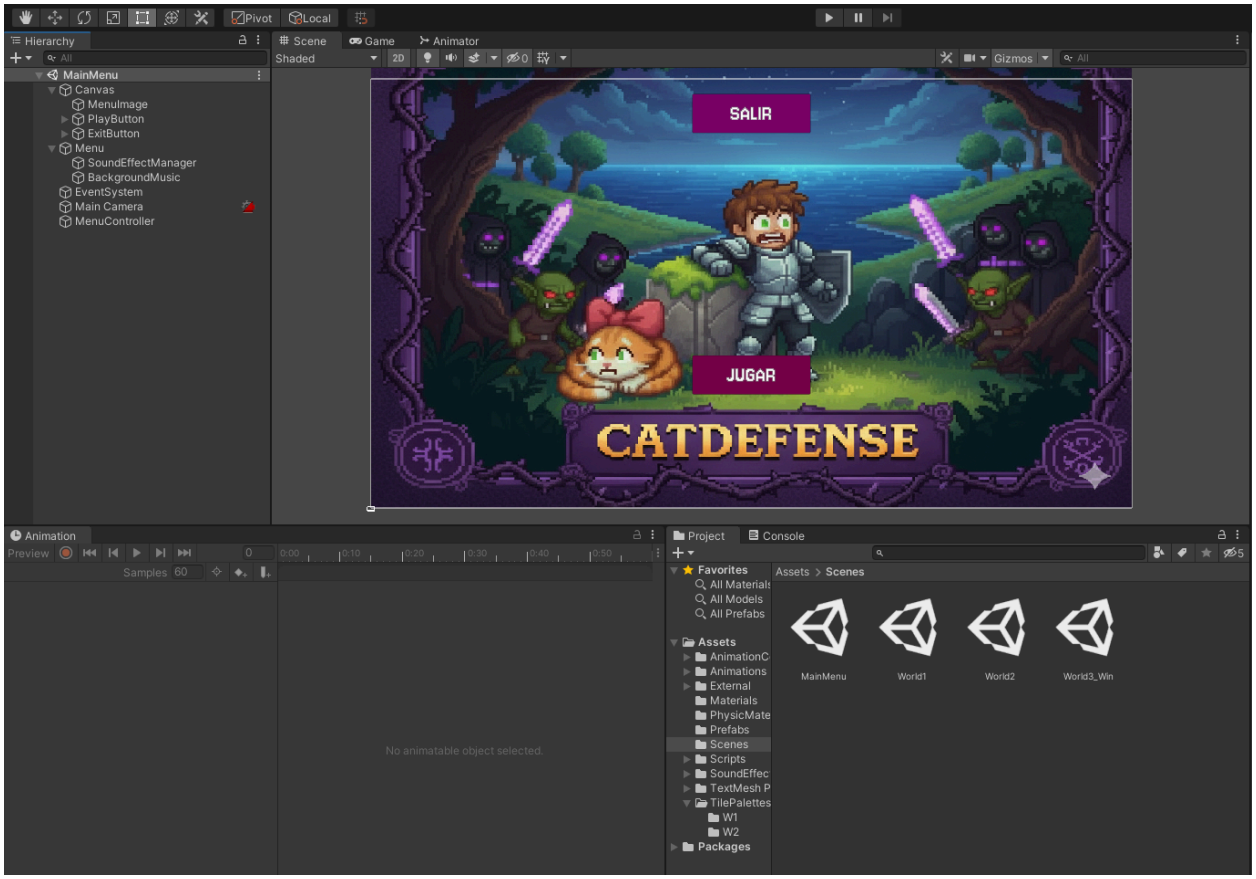
El mayor desafío de un GameManager persistente es mantener las referencias a objetos en los cambios de escena, se hizo por ello un registro dinámico:

1. Registro dinámico: en lugar de asignar la UI en el inspector, el GameManager expone métodos públicos como RegisterGameOverUI(GameObject uiPanel).
2. GameOverUI: este script, junto con LevelUIManager, vive en el canvas de cada escena, en su método Start se presenta al Singleton llamando a GameManager.instance.RegisterGameOverUI(panelContent). De esta manera, el GM siempre sabe a qué panel de escena actuar para activar.
3. ScoreUI: este script muestra un diseño desacoplado, cuando el evento OnScoreChanged se dispara, ScoreUI actualiza su texto.
4. Metas por nivel: para solucionar que scoreNextLevel tenga un valor único, se utilizó LevelUIManager. Al cargar cada escena, este script llama a una función de GameManager para informar de una nueva meta requerida (ej. 50 para Nivel 1, 200 para Nivel 2).

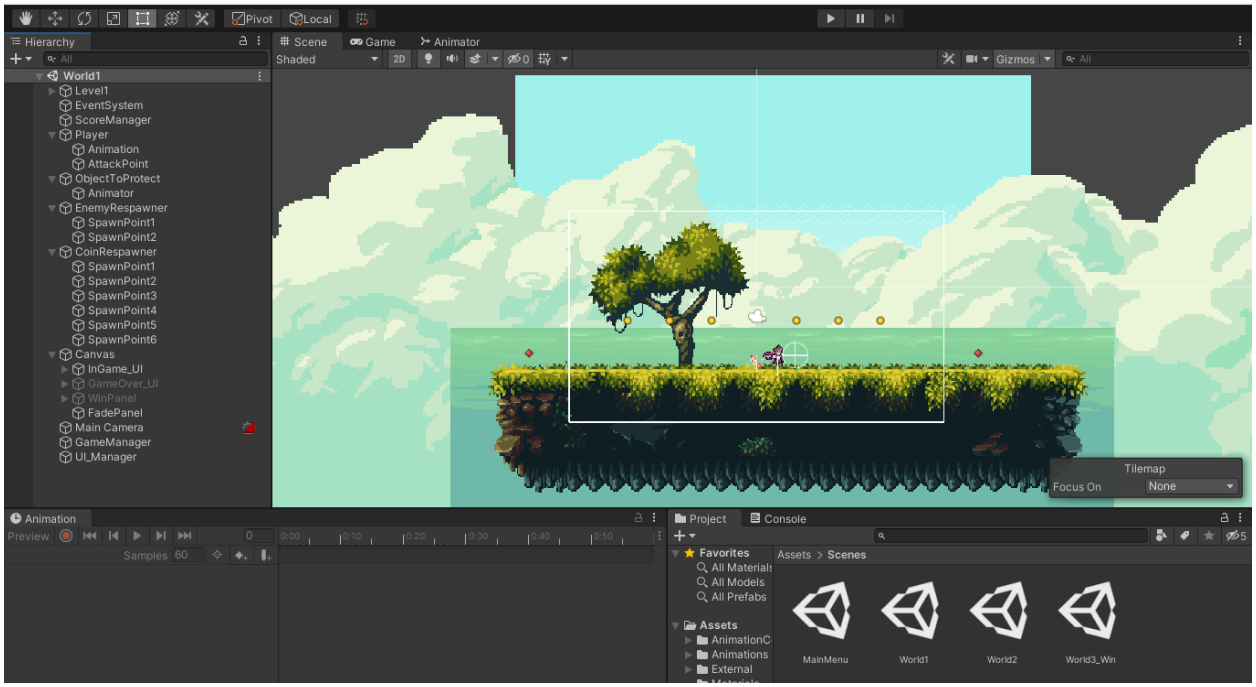
4.2. El ciclo completo.

La combinación de estos sistemas crea el flujo completo:

1. Escena 0 (MainMenu): dos botones con jugar o salir, el script MainMenu carga la escena 1.



2. Escena 1 (Nivel 1): se crean GameManager y ScoreManager, el LevelUIManager fija la meta en 50 puntos, el jugador recolecta monedas y protege al objetivo.



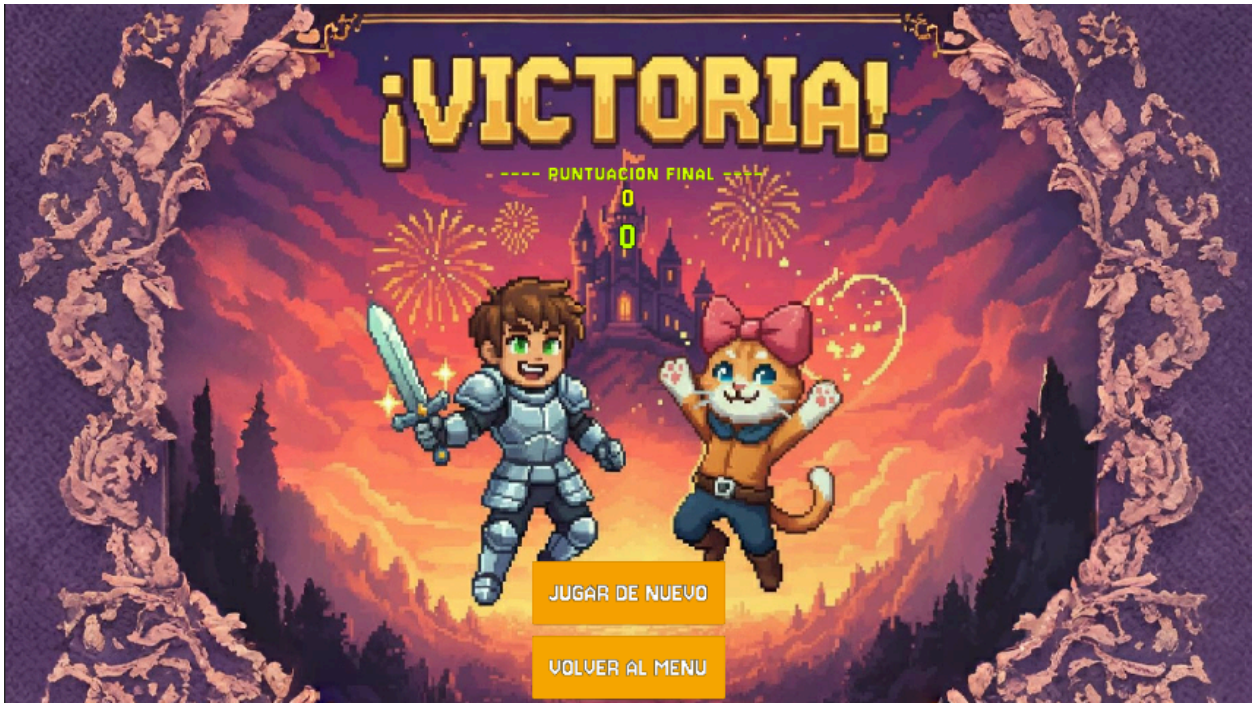
3. Transición: al alcanzar la meta, activa el panel de PressQ, el jugador cuando lo desee pulsa Q para cargar la escena 2.



4. Escena 2 (Nivel 2): se destruye la escena 1, pero GameManager y ScoreManager permanecen. El nuevo LevelUIManager establece la meta en 200 puntos, el GameManager resetea y carga escenas.



5. Final: al alcanzar la meta, se carga el panel PressQ, una vez pulsado, se carga la escena de victoria:
    a. Lee la puntuación final de ScoreManager.
    b. La compara con PlayerPrefs para establecer un récord.
    c. Limpia la partida destruyendo GameManager y ScoreManager.
    d. Ofrece botones para volver a jugar (cargar la escena 1) o salir al menú (cargar la escena 0).



6. Final 2 (Derrota): GameOverUI toma el control, ofrece un Restart o ReturnToMenu en 2 botones.



5. Pulido y resolución de problemas.

5.1. Bugs más notorios.

Durante el desarrollo surgen varios problemas que comprometían la jugabilidad.

1. El personaje abandonaba la pantalla por los laterales: para solucionarlo, se añadieron dos cajas como colliders delimitando la escena por los laterales.
2. Fallo de físicas en la Build: el juego funcionaba perfectamente en el editor, pero al crear la versión ejecutable, todos los personajes atraviesan el suelo. El problema residía en el Tilemap Collider 2D, el suelo estaba generando 169 formas de colisión separadas. El jugador se movía lo suficientemente rápido como para saltar las juntas microscópicas, por lo que se solucionó añadiendo un composite collider.
3. Bugs tras la solución física: el jugador y los enemigos giraban al atacar, el giro del player se basó en su xInput para solucionarlo, mientras que en Enemy se soluciona dando una nueva lógica donde se buscaba activamente la posición del jugador (antes el enemigo solo avanzaba mediante la velocidad sin tomar ningún target de inicio).
4. Bug de estado: si el jugador alcanzaba la puntuación de victoria, moría posteriormente, y su cadáver tocaba una moneda, el panel de victoria volvía a aparecer, bloqueando el juego. Para solucionarlo se implementó un bloqueo en GameManager que en cada OnScoreChanged se comprobaba si era Game Over (isGameOver).

5.2. Pulido y elementos de sonido.

Se implementó un SoundFXManager para gestionar de forma centralizada todos los efectos de sonido. Además, se añadió un BackgroundMusic como Singleton para gestionar la música ambiental y su posterior cambio de pitch durante el Game Over.

Se refinó el código de manera de que como principal objetivo fuera el desacoplamiento entre objetos para así crear nuevas escenas de la manera más sencilla posible.

En el caso de que quisieramos agregar un nuevo nivel, bastaría con hacer un copiapega del nivel 2 y cambiar música y escenografía, los enemigos podrían cambiarse de manera opcional, pero el juego pasaría de nivel correctamente sin ninguna clase faltante.

CONCLUSIONES

Este proyecto me ha servido como un ejercicio integral del desarrollo de juegos en 2 dimensiones, abarcando desde el manejo de scripts, managers y animaciones así como de la resolución de los problemas técnicos encontrados por el recorrido.

Aunque el proyecto tenga el flujo completo, se pueden añadir varias mejoras que tenía en mente pero que no ha sido posible debido al tiempo como:

1. Creación de un sistema de vida: que el jugador de alguna manera pueda obtener vida en los niveles.
2. Extensión del juego: añadir más niveles, escenas, elementos gráficos, sonidos...
3. Variación de arma / personajes: posibilidad de escoger personajes y armas.
4. Sistema de oleadas (Waves): enemigos apareciendo por oleadas, mejor manejo del sistema de puntuación.

He disfrutado haciendo este trabajo desde el minuto 1 y he aprendido conceptos clave del desarrollo en Unity como la herencia y el polimorfismo, además de conceptos clave como cómo funcionan las físicas, las escenas, la UI y las animaciones.